ISA 401: Business Intelligence & Data Visualization

05: Data Import II: Excel, JSON, and Export

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Open the project

GitHub Desktop → `business_intelligence` → **Fetch origin**. Then double-click **business_intelligence.Rproj**.
Top right corner of RStudio must say `business_intelligence` before you type anything.

3 Start today's markdown

File → New File → R Markdown..., title *Class 05*, **OK**. Delete everything below the setup chunk, then save as **markdowns/class05.Rmd**.
Same as last Wednesday. Your `class04.Rmd` stays as it is.

4 Pick your seat

Open your section's page (link on the Game 1 slide) and **tap the seat you are in**. Your seat will be used to represent you in two games in today's class.
You play on your own; helping a neighbor is allowed and encouraged.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Learning Objectives for Today's Class

- **Import** data from an **Excel workbook** and a **web-served JSON file**; run `dplyr::glimpse()` on each.
- **Read local files** with the `"../data/SOMETHING"` path and pull values out of imported data with the **three brackets**.
- **Export** results with `write_csv()` and `write_xlsx()`, then **knit and push** code, data, and output together.

PART 1	PART 2	PART 3
Find Your Seat	**Files on disk**	**Export**
Subsetting, live on the board	CSV, Excel, JSON, the Heist	Snapshots you can share

Part 1: Find Your Seat

Game 1: Find Your Seat

05:00

Section A

`isa401-find-your-seat.vercel.ap`

`/?section=A`

Open it in your browser on this computer.

Section B

`isa401-find-your-seat.vercel.ap`

`/?section=B`

Open it in your browser on this computer.

Copy

```
campus = jsonlite::fromJSON(
  "https://isa401-find-your-seat.vercel.app/campus.json")
```

One object, four brackets, your seat

0. Tap the seat you are in.
1. One element of `campus` is the business school. Its `code` field.
2. On the floor that holds room 2050, how many Fall 2026 sections meet? Its `sections` field.
3. How many Fall 2026 ISA sections meet in 2050?

Every answer is one subsetting line on `campus`. Wrong answers get a hint.

Look up. The projector shows this room. Your seat brightens as you clear each level and turns red when you arrive, with your finish rank.

5 / 20

One Table for Three Brackets

What comes back depends on the object and on the bracket. Read across a row for one object, down a column for one bracket.

Object	`[ ]`	`[[ ]]`	`$`
Vector `crashes_by_day`	a shorter **vector**	one element (rarely needed)	not allowed
List `example_list`, `job`	a shorter **list**	one element, whatever it is	one **named** element, same as `[["name"]]`
Data frame `crashes_df`	a smaller **data frame** `[rows, columns]`	one column as a **vector**	one column as a **vector**

The Pepper Shaker rule

- `[ ]` **keeps the container**: a smaller vector, list, or data frame of the same kind.
- `[[ ]]` and `$` **open the container**: you get what is inside.
- One bracket is still a shaker, two brackets is the packet. Say it that way and you will not forget it.

Part 2: Files on Disk

Excel: Download First, Then Read

1 Find the table in your browser. BLS Employment Projections → **Fastest growing occupations** → the **Other available formats: (XLSX)** link at the top.

2 Save it into your project. The file is `occupation.xlsx`. In the save dialog, browse to the **top level of your repo** (next to `markdowns`, not inside it), click **New Folder**, name it `data`, and save inside it.

3 Read it with a relative path.
`readxl::read_excel("../data/occupation.xlsx")`: from your markdown, up one level (`..`), then into `data/`. The next slide shows why that path works from Knit but not from the Console, then we read the file. The same line works on the lab PC, your laptop, and my machine when I grade.

Why the download this time?

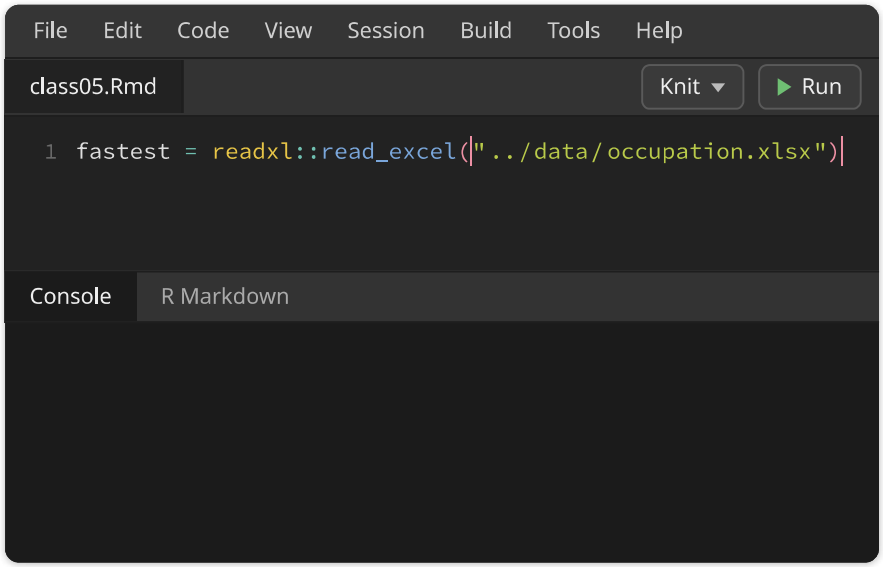
`read_excel()` **cannot read a URL**: a workbook has to be a file on disk.

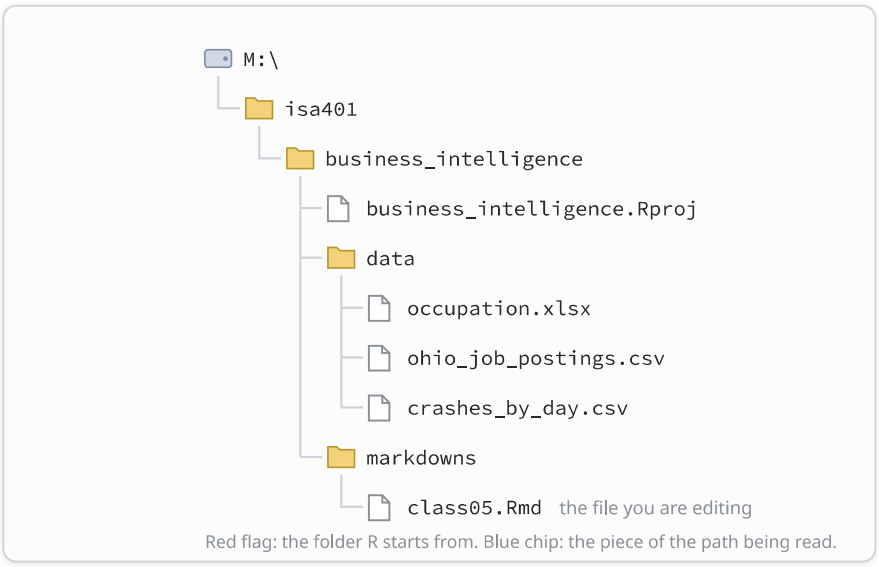
And BLS **turns away scripted downloads**: a browser gets the file, a script gets a 403 error page.

This is the project paying off

The file lives in `data/`, the code in `markdowns/`, both in the repo, both pushed. Whoever clones it can re-run everything with no path edits.

File Paths: Two Potential Starting Folders





1 Knit | 2 Run line | 3 The fix | 4 Any local file | Replay all

Play | Step | 0.5x | 1x | 2x

Reading an Excel File

```r
fastest = readxl::read_excel(
  "../data/occupation.xlsx",
  sheet = "Table 1.3",  # one of 12 sheets
  skip  = 1,            # title row above the header
  n_max = 31)           # stop before the footnotes
dplyr::glimpse(fastest)
```

```
Rows: 31
Columns: 7
$ `2025 National Employment Matrix title` <chr> "T…
$ `2025 National Employment Matrix code`  <chr> "0…
$ `Employment, 2025`                      <dbl> 17…
$ `Employment, 2035`                      <dbl> 17…
$ `Employment change, numeric, 2025-35`   <dbl> 59…
$ `Employment change, percent, 2025-35`   <dbl> 3.…
$ `Median annual wage, dollars, 2025`     <dbl> 50…
```

Paths in this course are always `"../data/SOMETHING"`: your markdown lives in `markdowns/`, your data in `data/`, so every local file you read is up one level, then into `data`.

Real workbooks are messy on purpose

`sheet`: this workbook has 12 sheets; we want Table 1.3, the fastest growing occupations.

`skip = 1`: row 1 is a title for humans, not a header.

`n_max = 31`: below the data sit three footnote rows that would otherwise pollute every column.

Data scientists are the third fastest growing occupation in the country: **+34.6%** by 2035, median wage **$120,230**. You are in the right room.

No package called readxl? It comes with tidyverse on the lab PCs but not on every laptop. Run `install.packages("readxl")` once in the Console, then re-run the chunk.

Source: Table 1.3, Fastest growing occupations, 2025 and projected 2035, U.S. Bureau of Labor Statistics, Employment Projections (modified Aug 27, 2026).

A Web-Served JSON File

```r
json_url = paste0(
  "https://data.cincinnati-oh.gov/",
  "resource/rvmt-pkmq.json?$limit=100")
crashes_json = jsonlite::fromJSON(json_url)
dplyr::glimpse(
  crashes_json[, c("crashdate", "dayofweek",
                   "crashseverity", "age")])
```

```
Rows: 100
Columns: 4
$ crashdate     <chr> "2026-08-22T22:29:00.000", "…
$ dayofweek     <chr> "SAT", "SAT", "SAT", "SAT", …
$ crashseverity <chr> "3 - MINOR INJURY SUSPECTED"…
$ age           <chr> "20", "48", "38", NA, "27", …
```

JSON is the shape of a list

The file holds an **array of records**, like `job` from Part 1, repeated 100 times. `fromJSON()` lines the records up: names become columns, records become rows, and the result is a data frame. A `.json` file on disk reads exactly the same way.

This one is served at a web address, so it is **current the moment you run the line**: the query after the `?` asks the city for the 100 most recent reports.

Every column is character, even the dates and ages: this file quotes every value. Monday's lesson applies: what a column looks like and what R stored are different things, and `glimpse()` is how you check. Cleaning types is a week 4 job.

Source: Traffic Crash Reports (CPD), City of Cincinnati Open Data, served as JSON.

Data Import for Cybersecurity Game

06:00

The Case | Answer | Discussion

INC-2026-0902: a 02:31 VPN sign-in from abroad.

Whose account? What was taken? The logs are **files at a URL**, and you are the analyst on duty. On your own, in a Rmd file:

1 **Identify the account.** Import the VPN sign-in log. Which account has the **most failed attempts** before a successful sign-in?

```
logins = readr::read_csv("https://isa401-import-heist.vercel.app/api/stage1")
```

2 **Scope the damage.** That account replaces **ACCOUNT** below. How many of the events in `log$events` are **DOWNLOAD**s?

```
log = jsonlite::fromJSON("https://isa401-import-heist.vercel.app/api/stage2?key=ACCOUNT")
```

3 **Close the ticket.** The count replaces **NUMBER** below. On **CLEARED**, read the report and post your team name on the **Menti wall**.

```
jsonlite::fromJSON("https://isa401-import-heist.vercel.app/api/finish?code=NUMBER")
```

Rules of engagement

1 **Open each URL in a browser tab first** and read the raw file. The evidence is small.

2 Then import it in R and `glimpse()` it. Use `[ ]`, `[[ ]]`, and `$` (Part 2) to pull the ACCOUNT out of stage 1 and the NUMBER out of stage 2.

3 Wrong key or code? **The system answers with a hint.** Read what comes back; it is data too.

The incident ticket

Briefing page: `isa401-import-heist.vercel.app`

Stage 1 is a CSV, so `read_csv()`. Stages 2 and 3 are JSON, so `fromJSON()`. Copy the calls on the left; only the **highlighted** parts change.

12 / 20

Part 3: Data Export

Writing Data Out

```r
readr::write_csv(ohio_postings,
  "../data/ohio_job_postings.csv")
writexl::write_xlsx(ohio_postings,
  "../data/ohio_job_postings.xlsx")
```

Copy

Writing mirrors reading: the same `"../data/SOMETHING"` path, in the other direction.

Why export at all?

Freeze a snapshot. The web-served files answer differently tomorrow; the file in `data/` is what you saw today.

Hand it off. Most of your teammates live in Excel; `write_xlsx()` meets them there.

Watch the **Files pane** as the chunk runs: `data/` now holds the downloaded workbook plus the two files you just wrote, all inside the project, all about to be committed.

Our Class Git + GitHub Ritual

1 Knit

Click **Knit**. `class05.html` re-runs every chunk top to bottom and saves the page next to your markdown.

If the knit fails, a chunk depends on something you only ran in the Console. Fix the markdown, not the Console.

→

2 Commit

GitHub Desktop lists today's work: `markdowns/class05.Rmd`, `class05.html`, and the `data/` files.

Summary: `Add class 05 markdown and data`.

→

3 Push and verify

Push origin, then check the repo on GitHub: `markdowns/` and `data/` both there, readable from any device.

That is code, data, and output, together.

If it is not pushed, it is not done. Your repo now carries code, data, and output that anyone, including future you, can re-run.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Import** an **Excel sheet** (`sheet`, `skip`, `n_max`) after downloading it into `data/`, and a **web-served JSON file** straight from its address.
- **Read** every local file as `"../data/SOMETHING"`, and use `[ ]`, `[[ ]]`, and `$` to pull one value out of what you imported.
- **Export** with `write_csv()` and `write_xlsx()`, then knit and push code, data, and output.

Skills practiced: the three brackets on a campus object and on incident logs, `glimpse()` on every import, reading the raw file in a browser before importing it.

Next class (Mon, Sep 14): a database file, one SQL query, and your first deployed app.

Before You Go

Join at **https://www.menti.com/al37nbpvnzpg**

Before you go: one thing from today you would like clarified.

Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

Assignment 04: Subsetting and Import

A Canvas quiz. Six answers come from subsetting `midwest_airbnb.db` in R, 14,887 Inside Airbnb listings from Chicago, Columbus, and the Twin Cities; four are about how today's imports work. Auto-graded, 10 points. Due date on Canvas.

Assignment 05: Deploy Your Midwest Airbnb Chat

Monday's recipe on the Midwest Airbnb listings: same Dockerfile, new data, a full data dictionary, extra instructions, your own theme and About section. Submit the Space's address. Monday's deck has the reading. Due date on Canvas.

Before you leave: knit, then commit and push `markdowns/class05.Rmd`, `class05.html`, and `data/`. If it is not pushed, it is not done.

Ungraded Practice Questions

1. Read `Table 1.4` (most new jobs) from `occupation.xlsx` with the same three arguments. How many rows, and what is the first occupation listed?

2. Change `$limit=100` to `$limit=500` in the Cincinnati address. How many rows come back?

3. From the Heist's stage 1 log, what is `max(logins$failed_attempts)`?

4. Export `fastest` to `../data/fastest_growing.xlsx` and open it in Excel.

5. Download `midwest_airbnb.db` from the Week 0 module into `data/`; Assignment 04's first question shows the three lines that read it.

▶ **Answers**